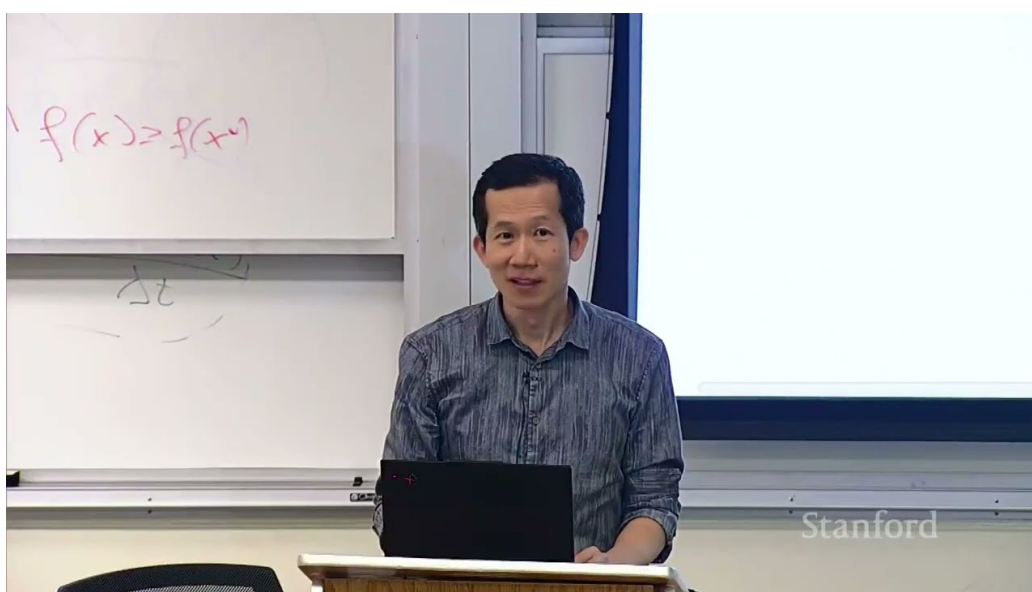


课程笔记

CS336 2026 Lecture 1: 从零构建语言模型、 课程版图与 Tokenization

基于 Stanford CS336 Spring 2026 官方可执行讲义整理

2026 年春季



课程/频道: Stanford CS336

发布日期: Spring 2026

材料类型: 官方可执行讲义 lecture01-slides.py

课程主页: <https://cs336.stanford.edu/>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 本讲定位：为什么 CS336 还要从零构建语言模型	2
1.1 API 抽象为什么不够	2
1.2 什么是 executable lecture	3
1.3 本章小结	3
2 工业化与尺度：frontier model 为什么不能直接复现	3
2.1 尺度会改变系统瓶颈	5
2.2 尺度也会改变能力表现	5
2.3 三类可迁移知识	6
2.4 Bitter Lesson 的正确读法	7
2.5 本章小结	7
3 语言模型发展版图：从 Shannon 到 agents	7
3.1 Pre-neural: Shannon 熵、n-gram 与困惑度	7
3.2 Neural ingredients: 术语不是名单，而是问题-机制对	9
3.3 Foundation models、scaling、open models	9
3.4 语言模型对象的演化	10
3.5 本章小结	10
4 课程机制：作业、AI policy 与学习方式	10
4.1 本章小结	11
5 课程版图：五个单元如何串起来	11
5.1 Scaling laws 与 data 的视觉例子	15
5.2 本章小结	16
6 Tokenization: 语言模型的输入接口	16
6.1 三种朴素 tokenizer	17
6.2 BPE: 从 bytes 到常见片段	18
6.3 Assignment 1	18
6.4 本章小结	19
7 总结与延伸	19
7.1 拓展阅读	19

1 本讲定位：为什么 CS336 还要从零构建语言模型

Lecture 1 是整门 CS336 的路线图。它没有直接进入 Transformer 公式，而是先回答一个更根本的问题：当 GPT、Claude、Gemini 这类 API 已经非常强时，为什么研究者还要自己实现 tokenizer、Transformer、optimizer、training loop、kernel、parallelism 和 inference stack？

课程给出的答案是：抽象层提高了生产力，但 LLM 的抽象层仍然非常漏。Prompt、tokenization、context length、sampling、latency、cost、failure mode、tool use、alignment 都会把底层机制重新暴露出来。只会调用 API 的研究者可以很快构建 demo，但很难回答系统为什么失败、如何扩展、如何调试、如何做真正新的研究。



Percy Liang
Instructor



Tatsunori Hashimoto
Instructor



Marcel Rød
CA



Herman Brunborg
CA



Steven Cao
CA

图 1: CS336 Spring 2026 课程团队。本讲开场强调这是第三次开课，课程仍坚持 from-scratch philosophy。

来源：CS336 Spring 2026 Lecture 1 官方可执行讲义，`welcome()`。

读图：课程团队页为什么重要

这张图不是普通封面。它告诉读者：CS336 是一门持续迭代的研究工程课程。2026 版仍保留“从零构建”的主线，但增加更多现代 LM ingredients，如 mixture of experts、long-context、agents。也就是说，本课不是复古地重写一个小 Transformer，而是用可控规模理解现代大模型系统。

第一讲的核心问题

在固定资源下，如何训练出最好的语言模型？这里的资源不只是 GPU FLOPs，还包括数据、显存、通信带宽、工程时间、推理预算和可调试性。第一讲所有内容都围绕这个问题展开。

1.1 API 抽象为什么不够

讲义把研究者和底层技术的距离变化压缩成三步：

阶段	常见工作方式	能力与局限
2016	自己实现并训练模型	能理解机制，但规模较小、工程成本高。
2018	下载 BERT 等模型并 fine-tune	生产力提升，但许多底层选择已经被预设。
今天	调用 GPT/Claude/Gemini API	应用构建更快，但很多 failure mode 无法解释。

只会使用 API 的局限

API 使用者很难回答这些问题：为什么 tokenizer 改变会影响验证集 loss？为什么同样参数量的模型训练速度不同？为什么长上下文推理突然变慢？为什么某些 prompt 的失败无法用“模型不够聪明”解释？这些问题都要求理解底层 stack。

1.2 什么是 executable lecture

CS336 2026 的 Percy 讲义经常以可执行 Python source 发布。‘text(...)’、‘image(...)’、‘link(...)’不是普通 slide 标记，而是构成 lecture trace 的节点；‘@inspect’、‘@stepover’让学生看到代码执行中的变量状态。

Listing 1: executable lecture 的最小心智模型

```
1 def lecture():
2     text("Explain an idea")
3     image("diagram.png")
4     value = compute() # @inspect value
5     text("Interpret the result")
```

为什么 executable lecture 适合 CS336

传统 slide 展示结论，代码展示实现，讲者口头解释连接二者。Executable lecture 把这三件事放进同一个 trace：可以读结构、看图、跑代码、检查变量。这与“understanding via building”的课程哲学一致。

1.3 本章小结

CS336 的第一讲先建立学习姿态：不要把 LLM 当远程 API，也不要小模型当玩具。课程希望学生在可控规模下掌握可迁移的 mechanics 和 mindset，然后谨慎处理 scale-specific intuition。

2 工业化与尺度：frontier model 为什么不能直接复现

语言模型已经工业化。GPT-4 级别模型训练成本据称达到上亿美元；xAI 等公司建设数十万 GPU 集群。这些事实改变了教学策略：课堂不能靠复现 frontier run 学习，而要靠小规模实验、资源核算和公开模型证据训练判断力。



图 2: 工业化隐喻: 前沿语言模型训练已经是大型基础设施工程。

来源: CS336 Spring 2026 Lecture 1 官方可执行讲义, `why_this_course_exists()`, 原图来自 Wikimedia Commons。

读图: 工业化图到底说明什么

图中是传统工业生产场景。它在这里对应 LLM 的集群、电力、网络、数据、冷却、工程团队和资本投入。模型质量不再只由一个算法公式决定, 而是由整套生产系统决定。后续 systems、scaling laws、data 单元都在拆这个生产系统。

闭源 frontier model 的另一个教学障碍是细节不可见。GPT-4 technical report 明确隐藏了模型大小、硬件、训练 compute、数据构成和训练方法等关键细节。

2 Scope and Limitations of this Technical Report

This report focuses on the capabilities, limitations, and safety properties of GPT-4. GPT-4 is a Transformer-style model [39] pre-trained to predict the next token in a document, using both publicly available data (such as internet data) and data licensed from third-party providers. The model was then fine-tuned using Reinforcement Learning from Human Feedback (RLHF) [40]. **Given both the competitive landscape and the safety implications of large-scale models like GPT-4, this report contains no further details about the architecture (including model size), hardware, training compute, dataset construction, training method, or similar.**

We are committed to independent auditing of our technologies, and shared some initial steps and ideas in this area in the system card accompanying this release.² We plan to make further technical details available to additional third parties who can advise us on how to weigh the competitive and safety considerations above against the scientific value of further transparency.

图 3: GPT-4 technical report 对训练细节的隐藏, 说明 frontier model 已无法作为完整可复现实验对象。

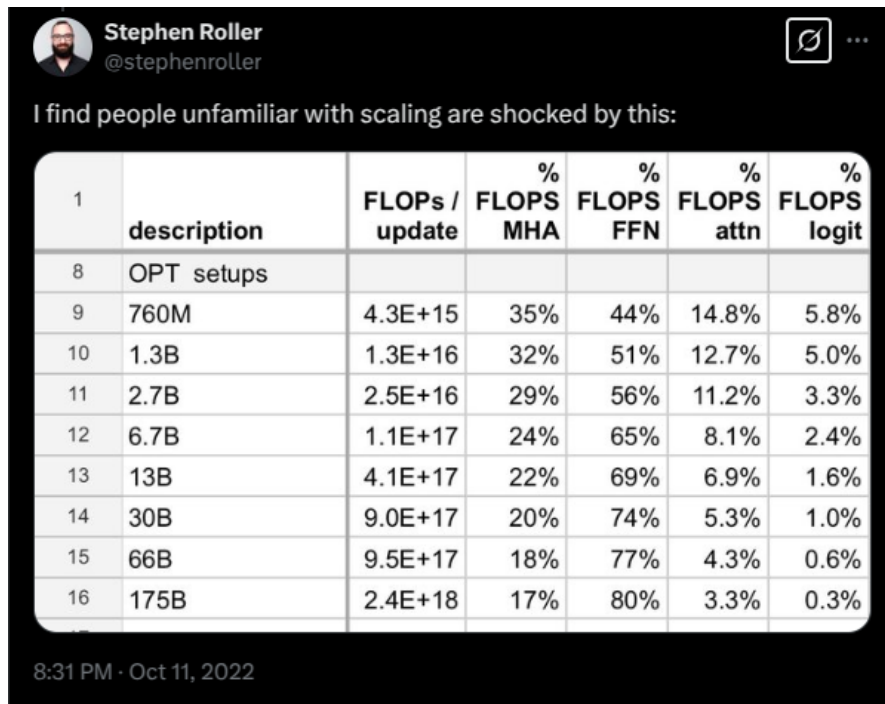
来源: CS336 Spring 2026 Lecture 1 官方可执行讲义, `why_this_course_exists()`。

读图时容易犯的错误

这张图不是在讨论开放/闭源的价值判断, 而是在说明课程约束: 如果 frontier recipe 不透明, 学生就不能通过照抄 recipe 学到完整技术栈。更可靠的学习目标是 mechanics、resource accounting、scaling mindset 和实验设计。

2.1 尺度会改变系统瓶颈

课堂能训练小于 1B 参数的模型，但这不代表小模型结论能直接外推。随着模型变大，FLOPs 分布、显存瓶颈、通信瓶颈、batch shape、推理成本都会变。



Stephen Roller (@stephenroller)

I find people unfamiliar with scaling are shocked by this:

1	description	FLOPs / update	% FLOPs MHA	% FLOPs FFN	% FLOPs attn	% FLOPs logit
8	OPT setups					
9	760M	4.3E+15	35%	44%	14.8%	5.8%
10	1.3B	1.3E+16	32%	51%	12.7%	5.0%
11	2.7B	2.5E+16	29%	56%	11.2%	3.3%
12	6.7B	1.1E+17	24%	65%	8.1%	2.4%
13	13B	4.1E+17	22%	69%	6.9%	1.6%
14	30B	9.0E+17	20%	74%	5.3%	1.0%
15	66B	9.5E+17	18%	77%	4.3%	0.6%
16	175B	2.4E+18	17%	80%	3.3%	0.3%

8:31 PM · Oct 11, 2022

图 4: 不同模型尺度下 attention、FFN、logit 等模块的 FLOPs 占比会变化。

来源: CS336 Spring 2026 Lecture 1 官方可执行讲义, 图片 `images/roller-flops.png`。

读图: Roller FLOPs 表应该怎么看

每一行是一个 OPT 模型尺度, 例如 760M、1.3B、13B、175B。列中 MHA 表示 multi-head attention 成本, FFN 表示 feed-forward network 成本, attn 更聚焦 attention score/value mixing, logit 是输出词表投影。关键趋势是: 模型越大, FFN 占比越高, logit 和 attention 的相对占比下降。它说明 “Transformer” 不是固定资源画像; 同一架构名下的真实瓶颈会随尺度漂移。

FLOPs 占比不是 wall-clock 时间占比

FLOPs 少的模块仍可能因为 memory bandwidth、kernel launch、通信、KV cache 读写而拖慢系统。读这张图时要区分 compute accounting 和 runtime profiling。Lecture 2 会建立这两者之间的桥。

2.2 尺度也会改变能力表现

讲义还引用 emergence 相关图。许多任务在小规模区域贴近 random baseline, 跨过某个训练规模后指标突然上升。它的教学信号不是 “大模型有魔法”, 而是: 小规模实验看到的能力曲线可能不完整。

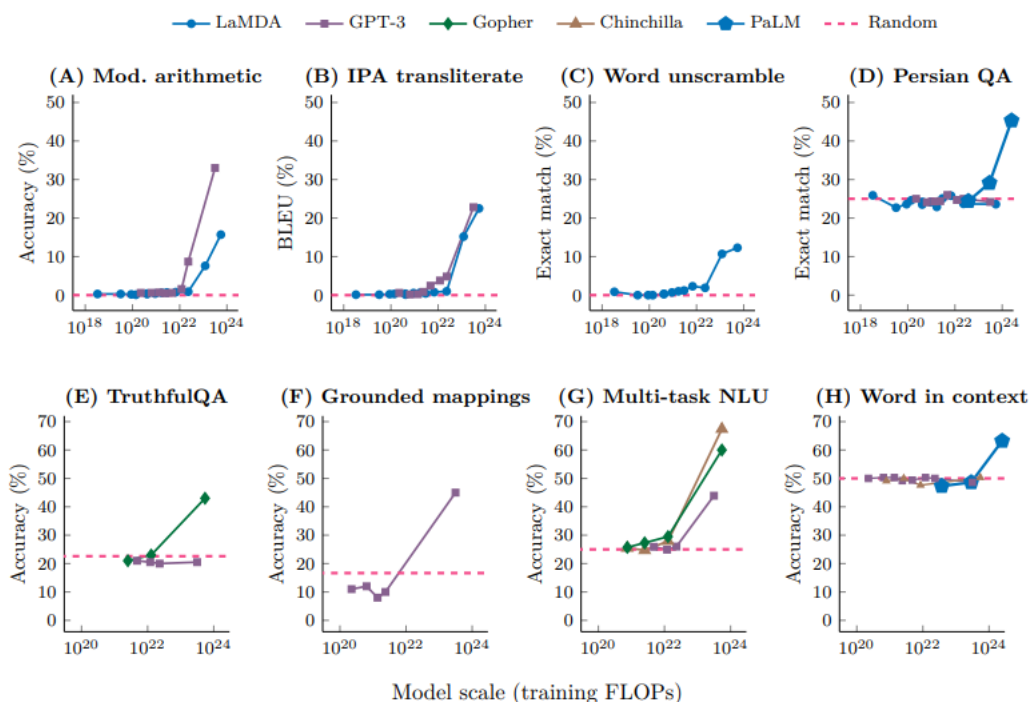


图 5: 一些任务指标随 training FLOPs 出现陡峭上升, 提示小模型直觉不能无脑外推。

来源: CS336 Spring 2026 Lecture 1 官方可执行讲义, 图片 `images/wei-emergence-plot.png`。

读图: emergence 曲线应该怎么看

每个小图是一项任务, 横轴是 training FLOPs 且为 log scale, 纵轴是 accuracy、BLEU 或 exact match。不同颜色是不同模型族, 粉色虚线是 random baseline。许多曲线先长期贴着 baseline, 然后在某个规模附近明显上升。这提醒我们: 评测指标可能像阈值函数, 模型能力也可能在小规模实验中不可见。

emergence 图不该被过度解读

陡峭上升不一定证明模型内部突然长出全新模块。它可能受指标离散化、prompt 格式、样本难度、横轴取对数、模型族差异影响。对 CS336 来说, 真正结论是: mechanics 和 mindset 可迁移, intuition 需要谨慎外推。

2.3 三类可迁移知识

第一讲把知识分成 mechanics、mindset、intuitions:

类型	含义	迁移性
Mechanics	Transformer、BPE、model parallelism、optimizer state、KV cache 等机制如何工作	高, 可以在课堂规模实现和检查。
Mindset	如何做 resource accounting、如何压榨硬件、如何认真对待 scaling	高, 是跨模型版本的工程习惯。
Intuitions	哪些数据、架构、训练技巧通常有效	部分迁移, 依赖尺度、数据和系统上下文。

4 Conclusions

We have extended the GLU family of layers and proposed their use in Transformer. In a transfer-learning setup, the new variants seem to produce better perplexities for the de-noising objective used in pre-training, as well as better results on many downstream language-understanding tasks. These architectures are simple to implement, and have no apparent computational drawbacks. We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.

图 6: Noam Shazeer 对 SwiGLU 的表述提醒我们：一些成功设计首先来自实验，而不是事前完备理论。

来源：CS336 Spring 2026 Lecture 1 官方可执行讲义，`why_this_course_exists()`。

读图：这句玩笑背后的方法论

SwiGLU 是后来许多 LLM 采用的 MLP 激活变体。图中文字的幽默点是：讲者承认某些设计最初不是由漂亮理论推出，而是由实验发现。课程要教的不是神秘化 intuition，而是如何设计实验、解释结果，并判断结论能否迁移。

2.4 Bitter Lesson 的正确读法

课程把 The Bitter Lesson 的误读拆开：错误读法是“规模就是一切，算法不重要”；正确读法是“能随着资源增长持续受益的算法最重要”。

$$\text{accuracy} = \text{efficiency} \times \text{resources}.$$

- accuracy：模型效果，可用 loss、held-out likelihood、benchmark 或真实任务质量衡量。
- efficiency：单位资源转化为有效能力的效率，包含算法、系统、数据和调参效率。
- resources：数据、训练 FLOPs、显存、通信带宽、工程时间和推理预算。

Bitter Lesson 对 CS336 的意义

规模放大了效率的重要性。课堂小模型浪费一点算力只是慢几分钟；frontier-scale 训练中同样比例的浪费可能意味着数百万美元、数周排队和错过实验窗口。

2.5 本章小结

工业化和尺度共同决定了 CS336 的路线：课堂不能复现 frontier model，但能训练可迁移的实现能力、资源意识和实验判断。小模型是学习机制的实验台，不是前沿模型的完美缩小版。

3 语言模型发展版图：从 Shannon 到 agents

3.1 Pre-neural: Shannon 熵、n-gram 与困惑度

早期 language model 主要服务于英语熵估计、机器翻译和语音识别。Shannon 的语言熵工作说明：预测下一个符号本身就是理解语言统计结构的一种方式。N-gram 则用固定长度上下文和计数来估计序列概率。

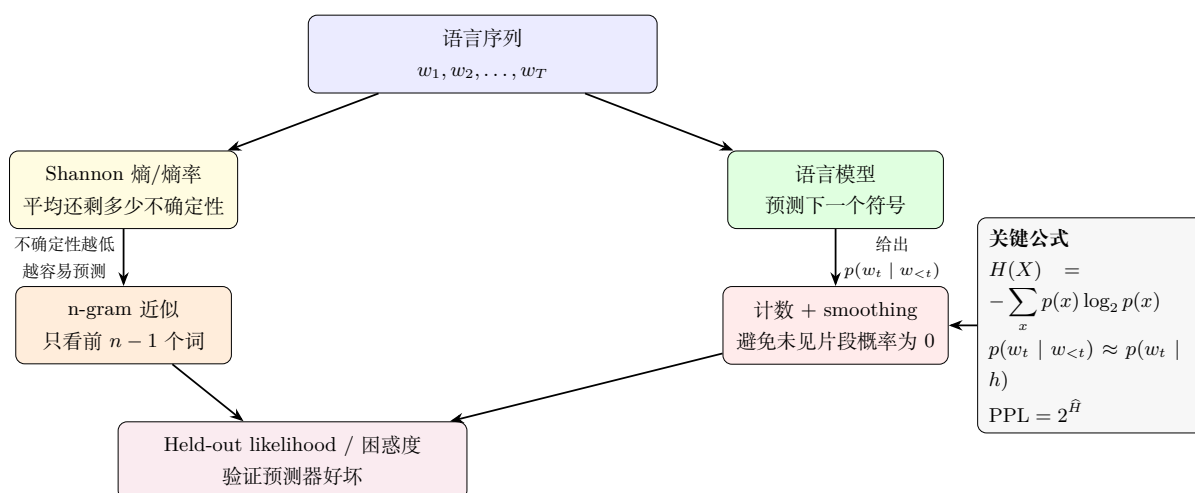


图 7: 从 Shannon 熵到 n-gram: 早期语言模型把语言结构落成“预测下一个符号并在 held-out 数据上评估”。

背景概念: Shannon 语言熵

熵衡量不确定性。若看到前文后越容易猜出下一个字符或词, 语言熵率就越低。语言模型的 next-token prediction 不是文字游戏, 而是在测模型能否捕捉语言中的冗余、语法、语义和世界知识线索。

背景概念: n-gram

n-gram 是连续 n 个符号组成的片段。n-gram 语言模型假设预测 w_t 时只看前 $n - 1$ 个词:

$$p(w_t | w_1, \dots, w_{t-1}) \approx p(w_t | w_{t-n+1}, \dots, w_{t-1}).$$

它用计数估计条件概率。问题是未见片段会得到 0 概率, 所以需要 smoothing、backoff 或 interpolation。

背景概念: perplexity 的直觉

Perplexity (困惑度, 常缩写为 PPL) 通常写作 $\text{PPL} = 2^{\hat{H}}$ 或 $\exp(H)$, 其中 H 是平均交叉熵。直觉上, PPL 约等于模型在每个 token 位置“等效面对多少个均匀候选”。 $\text{PPL} = 1$ 表示完美预测; PPL 接近词表大小表示近似瞎猜。这个直觉有用, 但 PPL 不等价于有用性、安全性或推理能力。

perplexity 的边界

Perplexity 是语言建模 held-out likelihood 的变体, 适合比较同一 tokenizer、同一数据分布上的 next-token prediction。不同 tokenizer、不同数据清洗方式、不同评测集污染程度都会改变 PPL 的可比性。它能告诉我们模型是否更会预测文本, 却不能直接告诉我们模型是否更会推理、更安全或更会调用工具。

3.2 Neural ingredients: 术语不是名单，而是问题-机制对

2010 年代的关键积木不是孤立论文名，而是围绕三个瓶颈展开：表示稀疏、序列信息路由、系统扩展。

术语	解决的问题	核心机制与课程关系
LSTM	RNN 难以长期保存信息	用 gates 控制 cell state 写入、遗忘、读取；证明可学习记忆重要，但串行结构不利于大规模并行。
Neural LM	n-gram 稀疏和未见组合泛化差	用连续向量表示词和上下文，让相似片段共享统计强度。
Seq2seq	传统 MT 依赖手工特征 pipeline	encoder 编码输入、decoder 生成输出，把翻译写成端到端条件生成。
Attention	固定向量瓶颈丢失长句信息	decoder 动态查看输入位置；后来发展为 self-attention。
Adam	深层网络训练对学习率敏感	用一阶/二阶动量自适应缩放更新，降低调参难度。
Transformer	RNN 串行限制并行训练	self-attention + MLP + residual/norm，成为现代 LLM 主干。
MoE	dense 模型每 token 激活全部参数，计算贵	router 只激活少量 experts，实现大参数量和低每 token compute 的折中。
GPipe	单设备放不下深模型	按 layers 切 pipeline，用 microbatch 降低 bubble。
ZeRO	data parallel 下 optimizer state、grad、param 重复存储	ZeRO 即 Zero Redundancy Optimizer：在 DP ranks 之间分片这些训练状态。ZeRO-1 切 optimizer state，ZeRO-2 再切 gradients，ZeRO-3 连 parameters 也切；stage 越高越省显存但通信越多。
Megatron-LM	单层矩阵太大，单卡算不动	用 tensor parallelism 切矩阵乘法，并与 data/pipeline parallelism 组合。

first-use glossary: sharding 与 state sharding

Sharding 就是分片：原本每张 GPU 都保存完整对象，分片后每张 GPU 只保存一部分。State sharding 特指把训练状态分片，例如 optimizer state、gradient、parameter。它不是让单卡独立完成完整 forward，而是在通信配合下减少重复显存。

3.3 Foundation models、scaling、open models

ELMo、BERT、T5 展示了预训练再适配；GPT-2、GPT-3、PaLM、Chinchilla 把 scaling laws 推到中心；Llama、Mistral、DeepSeek、Qwen、Olmo、Nemotron、Marin 等开放模型让外部研究和教学能看到更多 recipe。

阶段	代表材料	给 CS336 的启发
----	------	-------------

Early foundation models	ELMo、BERT、T5	预训练成为通用能力来源，fine-tuning/text-to-text 改变任务形式。
Scaling era	GPT-2、GPT-3、PaLM、Chinchilla	用小实验预测大训练，用 compute budget 决定参数量和 token 数。
Open-weight	Llama、Mistral、DeepSeek、Qwen 等	权重可用，论文可读，但数据/code/recipe 未必透明。
Open-source/open development	Olmo、Nemotron、Marin	更适合教学和可复现研究，因为能看到数据、代码和过程。

open-weight 不等于 open-source

Open-weight 只说明权重可下载；open-source/open-development 更强调数据、代码、训练 recipe、评估和开发过程透明。CS336 需要后者，因为本课关心“模型如何被造出来”。

3.4 语言模型对象的演化

语言模型的“使用对象”也在变化：2018 年的 BERT 是 fine-tune 的模型；2020 年的 GPT-3 是 prompt 的模型；2022 年的 ChatGPT 是对话对象；2026 年的 agent 则是会自主行动、调用工具、写代码、执行任务的系统组件。

fundamentals 相同，specs 不同

底层仍是 attention、kernels、optimization、data 和 scaling；但 specs 变了：context 更长，inference efficiency 更重要，模型从生成一句话变成执行多步任务。课程后续内容都会围绕这些 specs 展开。

3.5 本章小结

语言模型历史不是“从 n-gram 到 GPT”的线性故事，而是概率建模、表示学习、可并行架构、系统扩展、开放生态和 agentic use case 的叠加。

4 课程机制：作业、AI policy 与学习方式

CS336 是 5-unit 课程，作业强度非常高。讲义引用 Spring 2024 评价说，第一份作业接近 CS224N 五份作业加 final project 的工作量。这不是为了堆工作，而是为了让学生真正实现和 benchmark。

作业	内容	单元
Assignment 1	BPE tokenizer、Transformer、cross-entropy、AdamW、training loop、TinyStories/OpenWebText leaderboard	Basics
Assignment 2	fused RMSNorm kernel、distributed data parallel、optimizer state sharding、benchmark/profile	Systems
Assignment 3	小规模训练 API、fit scaling laws、预测大规模 loss/超参	Scaling

Assignment 4	Common Crawl 到文本、filtering、dedup、data mixing、Data perplexity leaderboard	
Assignment 5	DPO、GRPO、偏好优化和 RL systems	Alignment

AI policy 的真实含义

Coding agents 可能完成作业，但如果学生只拿答案，就绕过了课程目标。AI 可以用来解释、提问、debug，但不能替代理解 mechanics。讲义生成也一样：不能只给结论，要展示机制和推理。

4.1 本章小结

课程机制服务于学习目标：通过实现、测试、benchmark 和资源核算建立理解。作业不是普通 coding exercise，而是逐步搭建语言模型训练系统。

5 课程版图：五个单元如何串起来

Lecture 1 的 syllabus 把学期拆成 Basics、Systems、Scaling Laws、Data、Alignment。它们不是并列主题，而是从“能训练一个模型”逐步走向“在真实约束下训练、服务和改进模型”。

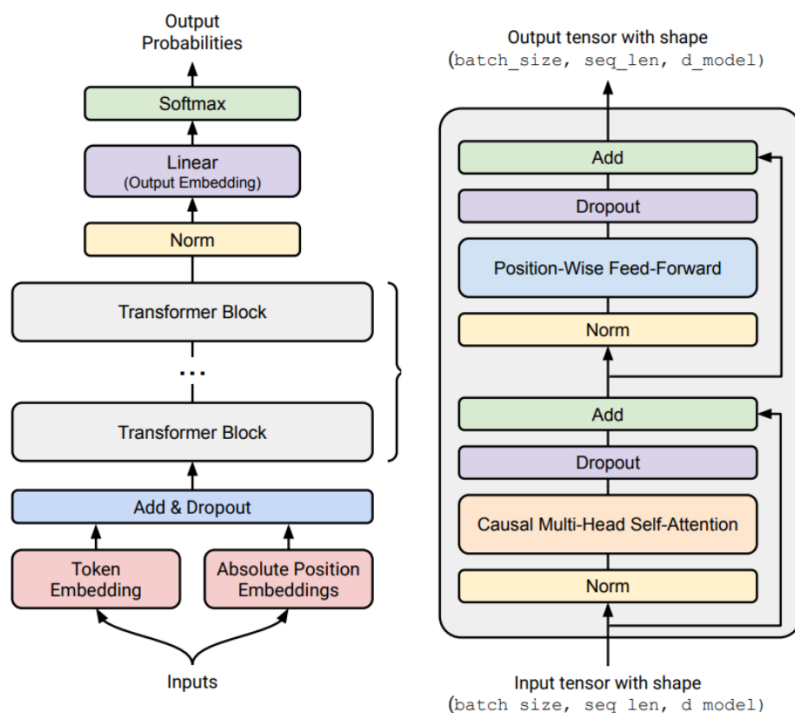


图 8: Transformer architecture 是 Basics 单元模型起点。后续会逐步拆 attention、MLP、normalization、position encoding 等组件。

来源: CS336 Spring 2026 Lecture 1 官方可执行讲义, `basics()`。

读图：Transformer 架构图在第一讲里的作用

这张图不是要在第一讲完整推导 Transformer，而是告诉读者后续要拆哪些对象：attention、feed-forward、residual、normalization、position information。From-scratch 的意思是这些模块都要能实现、记账、调试和替换。

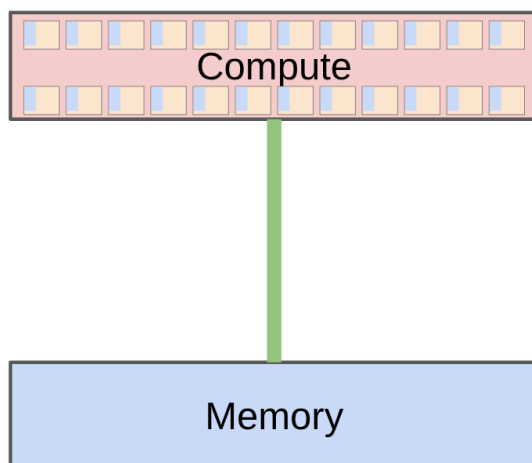


图 9: compute 与 memory 的基本关系贯穿 systems 和 resource accounting。

来源：CS336 Spring 2026 Lecture 1 官方可执行讲义，`systems()`。

读图：compute-memory 小图为什么反复出现

图中数据从 memory 进入 compute units，计算后再写回 memory。它会在 Lecture 2 变成 arithmetic intensity，在 kernels 单元变成 fusion/tiling，在 inference 单元变成 KV cache 读写。模型效率不是只看 FLOPs，还要看数据移动。

first-use glossary: DRAM、SRAM、HBM

DRAM 是 Dynamic Random-Access Memory，动态随机存储器；GPU 上的大容量高带宽显存通常是 HBM，即 High Bandwidth Memory。SRAM 是 Static Random-Access Memory，静态随机存储器，常对应芯片内部高速缓存或片上存储。计算单元不能无限快地直接从 HBM 取数，数据通常要经过片上缓存/寄存器才能高效计算，因此“少搬数据”是 kernel 优化的核心。

first-use glossary: fused kernel 与 collectives

Fused kernel 是把多个 GPU 操作合并成一个 kernel，减少 HBM 读写和 kernel launch overhead。例如 RMSNorm 中的均方、缩放、乘权重可以尽量融合。Collectives 是多 GPU 集合通信原语，如 all-reduce、reduce-scatter、all-gather、broadcast；DDP 梯度同步常用 all-reduce，ZeRO/FSDP 会大量使用 reduce-scatter 和 all-gather。

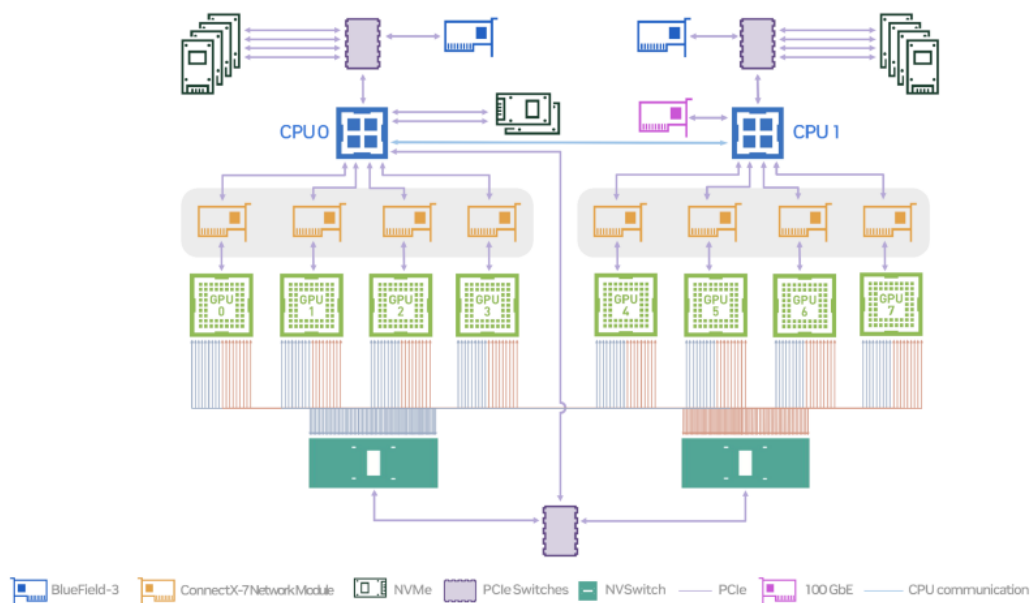


图 10: DGX B200 拓扑图提醒学生：真实训练不是抽象 FLOPs，而是 GPU、CPU、NVLink、NIC、storage 的组合系统。

来源：CS336 Spring 2026 Lecture 1 官方可执行讲义，`systems()`，图片来自 NVIDIA 文档。

读图：DGX 拓扑图说明什么

图中每张 GPU 都通过 NVLink/NVSwitch、CPU、NIC、storage 参与系统。数据并行、张量并行、流水线并行、optimizer state sharding、checkpointing 都会受到拓扑约束。真实训练不是“GPU 越多越快”，而是通信和计算要匹配。

并行方式	切分对象	单卡能否独立 forward	for-	通信代价与适用场景
Data parallelism	batch	能		低到中；主要同步梯度，适合模型能放进单卡但想提升吞吐。
Tensor parallelism	matrix / tensors	不能		高；每层可能通信，适合单层矩阵太大或要吃满多卡 tensor cores。
Pipeline parallelism	layers	不能		中；层边界传 activation，有 pipeline bubble，适合深模型跨设备放置。
Sequence parallelism	sequence dimension	不能		中；支撑长上下文，attention/norm 等需要跨序列通信。
Expert parallelism	MoE experts	部分不能		all-to-all 代价高，适合稀疏 MoE 扩大参数量。

并行方式可以组合

真实训练常用 3D/4D parallelism, 例如 data parallel $\times$ tensor parallel $\times$ pipeline parallel, 再叠加 sequence/expert parallelism。选择组合不是形式问题, 而是为了把参数、activation、optimizer state 和通信量放进硬件拓扑。

Systems 单元导读：为什么第一讲先讲这些词

第一讲只是预告 systems 单元, 但这些词会在后续反复出现。Kernel 解决单 GPU 内部“少搬 HBM、让 tensor cores 忙起来”的问题; parallelism 解决多 GPU 上参数、activation、gradient、optimizer state 放不下或算不快的问题; inference 解决模型训练好以后如何低延迟、高吞吐地生成 token 的问题。三者共同约束 architecture 选择: 例如 GQA 是架构改动, 也是推理 KV cache 优化; RMSNorm 是模型组件, 也是减少数据移动的系统优化。

并行表格的误读

Data parallelism 中“单卡能独立 forward”只在模型完整复制到每卡时成立; ZeRO-3/FSDP 会把参数也分片, 此时 forward 前需要 all-gather 参数。Tensor/pipeline/sequence/expert parallelism 也不是互斥选项, 而是经常组合成多维并行。第一讲的表格是地图, 不是完整实现说明。

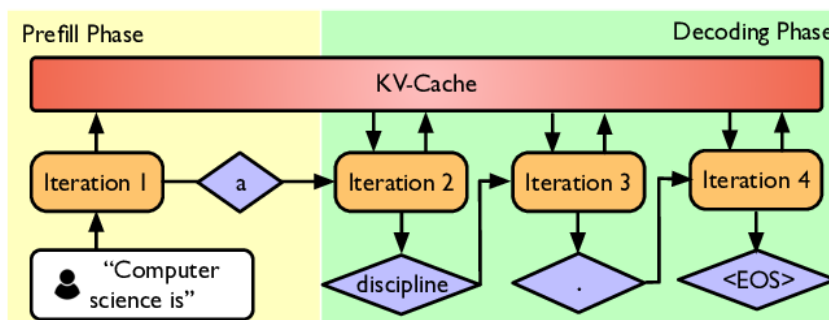


图 11: Inference 被拆成 prefill 与 decode; 两者并行性和瓶颈不同。

来源: CS336 Spring 2026 Lecture 1 官方可执行讲义, `systems()`。

读图: prefill/decode 为什么提前出现

Prefill 阶段 prompt tokens 已知, 可以并行处理, 形态接近训练; decode 阶段每次只能生成一个 token, 容易 memory-bound。推理不是简单 forward pass, 而是有独立系统瓶颈。

5.1 Scaling laws 与 data 的视觉例子

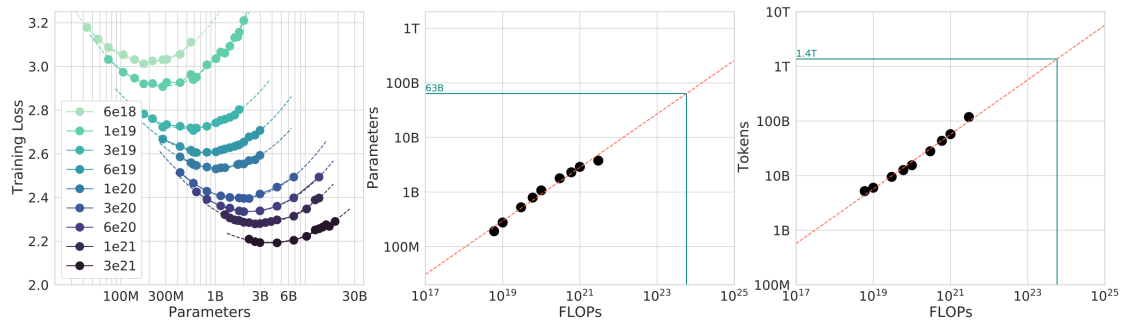


图 12: Chinchilla 风格 IsoFLOP 分析：在固定 compute 下寻找参数量与训练 token 数的平衡。

来源：CS336 Spring 2026 Lecture 1 官方可执行讲义，`scaling_laws()`。

读图：IsoFLOP 曲线怎么看

每条曲线对应一个固定 compute budget，曲线上不同点代表不同参数量和 token 数组合。最低点表示该预算下最优模型大小。多个预算的最优点连起来，可以外推到更大 compute。重点是 forecasting workflow，不是机械记住 $D = 20N$ 。

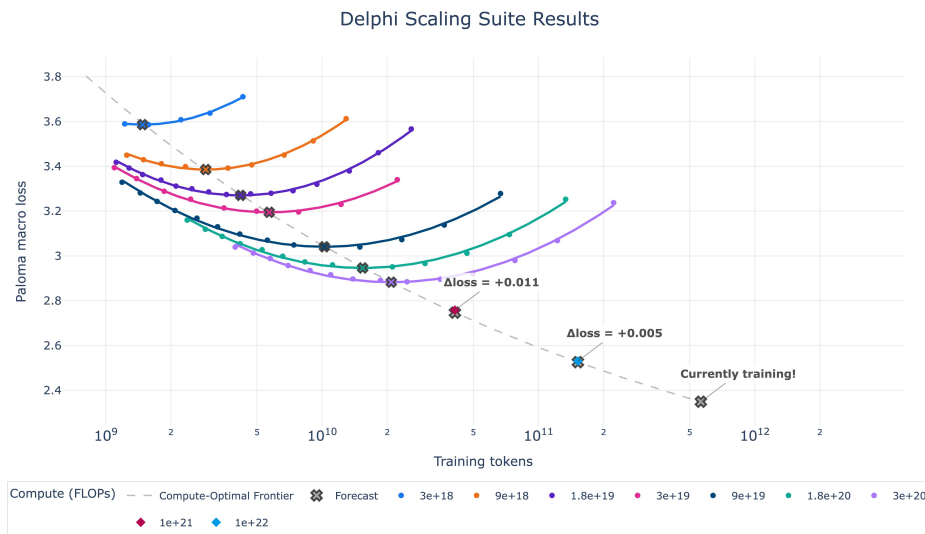


图 13: Marin live example 展示预注册 scaling 预测和实际训练 loss 的对照。

来源：CS336 Spring 2026 Lecture 1 官方可执行讲义，`scaling_laws()`。

读图：Marin 预测图的教学信号

这类图把 scaling laws 从论文公式变成训练前承诺。如果实际 loss 落在预测附近，说明小规模实验、超参迁移和数据假设比较可靠；若偏离很大，就要检查实现、数据、优化器和分布漂移。

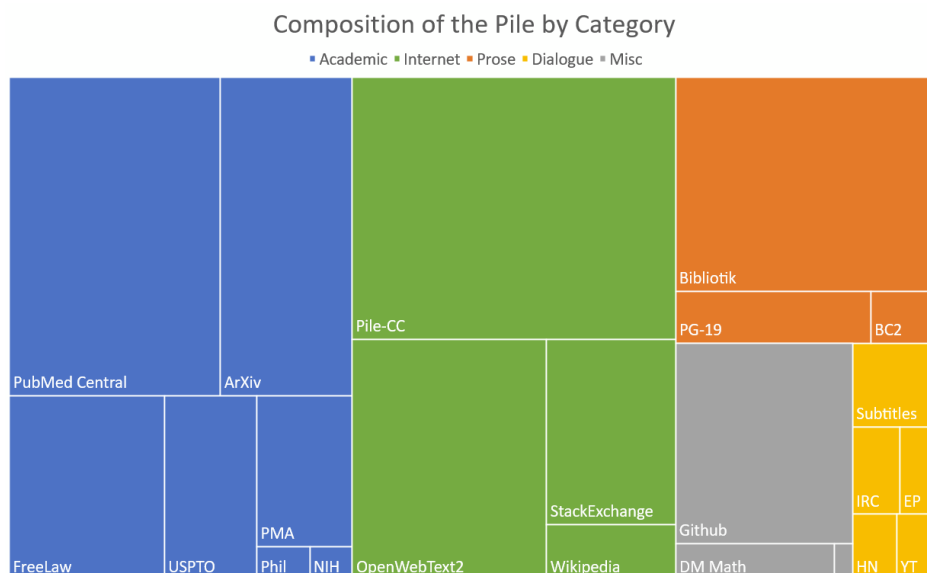


图 14: The Pile 数据构成展示预训练数据来自多种来源，而不是同质 token 池。

来源: CS336 Spring 2026 Lecture 1 官方可执行讲义, `data()`, 图片来自 *The Pile* 论文页面。

读图：数据混合图为什么重要

每个扇区对应一种数据来源。不同 token 消耗相近 FLOPs，但带来的学习信号不同。高质量代码、论文、书籍、网页噪声对模型能力的影响差别很大，所以 data curation 是效率问题。

5.2 本章小结

五个单元其实都在回答同一个问题：固定资源下如何最大化有效能力。Basics 让模型能训练，systems 让训练能跑快，scaling laws 让大训练可预测，data 让 compute 花在有价值 token 上，alignment 让模型行为符合目标。

6 Tokenization：语言模型的输入接口

语言模型不直接操作 Python string，而是操作整数 token 序列。Tokenizer 在 raw text/bytes 和 token IDs 之间转换。

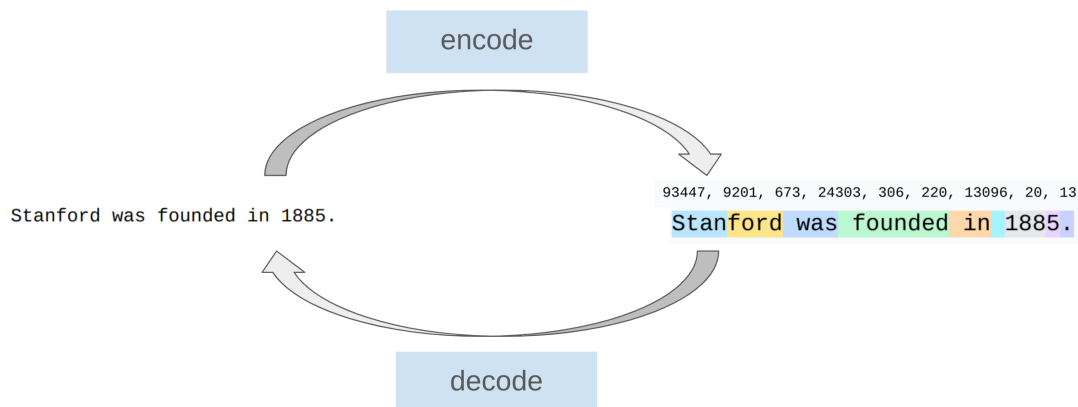


图 15: Tokenizer 把原始字符串切成 token，并把每个 token 映射为整数 ID。

来源: CS336 Spring 2026 Lecture 1 官方可执行讲义, `basics()`。

读图: tokenized example 说明什么

图中同一段文本被切成不同 token。Token 不一定是词: 可能包含前导空格、词片段、标点、emoji 或多字节字符。同一句文本用不同 tokenizer 会得到不同长度和 ID 序列, 因此 tokenizer 是模型 API 的一部分。

Listing 2: Tokenizer 抽象接口

```

1 class Tokenizer:
2     def encode(self, string: str) -> list[int]:
3         raise NotImplementedError
4
5     def decode(self, indices: list[int]) -> str:
6         raise NotImplementedError

```

6.1 三种朴素 tokenizer

方案	机制	优点	失败点
Character	Unicode code point	直观, 可 round-trip	词表大, 稀有字符浪费容量, 压缩率低。
Byte	UTF-8 bytes, 0-255	小固定词表, 永不 OOV	序列长, attention 成本高。
Word	regex/空格切词	token 有语义, 压缩率好	词表无限, 稀有词多, 需要 UNK。

UNK token 为什么麻烦

Word tokenizer 遇到未见词会映射成 UNK。不同未知词被压成同一个符号, 模型无法区分, perplexity 也会被扭曲。现代 LLM 更偏向 byte-level 或 subword 方法, 因为它们至少能表示任意字符串。

6.2 BPE: 从 bytes 到常见片段

Byte Pair Encoding 从 byte token 开始，反复合并最常见的相邻 token pair。常见片段变成短 token，稀有片段仍由多个 byte/subword 表示。

Listing 3: BPE merge 的核心逻辑

```
1 def merge(indices, pair, new_index):
2     new_indices = []
3     i = 0
4     while i < len(indices):
5         if i + 1 < len(indices) and (indices[i], indices[i+1]) == pair:
6             new_indices.append(new_index)
7             i += 2
8         else:
9             new_indices.append(indices[i])
10            i += 1
11     return new_indices
```

步骤	做什么	为什么重要
初始化	词表为 256 个 bytes	能表示任意 UTF-8 字符串，不需要 UNK。
统计 pair	统计相邻 token pair 出现次数	找出最值得压缩的局部片段。
merge	把最高频 pair 替换为新 token	缩短序列长度，降低 attention 成本。
重复	执行固定次数 merge	在词表大小和压缩率之间折中。
encode/decode	用 learned merges 编码，再按 vocab 还原 bytes	让 tokenizer 成为可复用协议。

BPE 为什么是强 baseline

BPE 比 byte tokenizer 短得多，比 word tokenizer 更开放，训练简单，部署稳定。它不是最终答案，但在有限 context 和有限 compute 下非常实用。

BPE 的 off-by-one 风险

合并一个 pair 后，指针必须跳过两个旧 token。否则会重复使用已经合并的 token，得到错误 tokenization。这类细节正是 assignment 1 要学生亲手实现的原因。

6.3 Assignment 1

Assignment 1 要求实现 BPE tokenizer、Transformer、cross-entropy loss、AdamW optimizer、training loop，并在 TinyStories 与 OpenWebText 上训练。Leaderboard 要在固定 B200 时间预算内最小化 OpenWebText perplexity。

Assignment 1 的真正目标

它不是让你“跑一个模型”，而是让你第一次把 tokenizer、architecture、loss、optimizer、training loop 和 resource accounting 连接起来。后续所有单元都建立在这条链上。

Training 超参数导读

第一讲列出的 optimizer、initialization scale、learning-rate schedule、regularization、batch size、MoE load balancing 都会在后续拆开。这里先给直觉：optimizer 决定梯度如何变成参数更新；initialization 决定训练开始时信号尺度是否健康；learning-rate schedule 决定不同阶段的更新强度；batch size 影响梯度噪声和硬件利用率；MoE load balancing 防止 router 把所有 token 都送到少数 experts。它们共同服务于三件事：expressivity、stability、efficiency。

perplexity leaderboard 不等于模型全能力

Assignment 1 的 leaderboard 用 OpenWebText perplexity，是因为它能稳定衡量 next-token prediction 的基本能力，也便于在固定时间预算下比较 recipe。但 PPL 低不代表模型更会推理、更会对话或更安全。后续 evaluation 和 alignment 单元会把“语言建模指标”和“真实任务质量”区分开。

6.4 本章小结

Tokenization 是语言模型的输入协议。Character、byte、word tokenizer 都有明显失败点；BPE 用数据驱动的 merge 在开放词表和序列长度之间取得折中。它看似预处理，却直接影响训练效率和模型行为。

7 总结与延伸

Lecture 1 建立了 CS336 的整体方法：

1. LLM 已经工业化，frontier model 昂贵且不透明，所以课堂要训练可迁移 mechanics 和 mindset。
2. 小模型能教实现和调试，但不能保证所有系统瓶颈和能力表现都能外推。
3. 现代语言模型技术栈来自概率建模、神经表示、Transformer、系统并行、scaling laws、数据工程和 alignment 的叠加。
4. Tokenizer 是第一块真正可实现的积木，它把文本变成模型可训练的离散序列。

最终 takeaway

From scratch 不是怀旧，而是研究方法。只有把 tokenizer、architecture、training loop、optimizer、hardware、data 和 inference 都拆开，才能知道模型质量来自哪里，也知道该从哪里改进。

7.1 拓展阅读

- Shannon 1950: entropy and prediction of English.
- Sennrich et al. 2016: Byte Pair Encoding for neural machine translation.
- Vaswani et al. 2017: Transformer.
- Kaplan et al. 2020 / Chinchilla 2022: scaling laws and compute-optimal training.
- JAX Scaling Book: systems and resource accounting for large models.